

Rapport Vulnérabilités

Mathéo Gros, Mathieu Piekarz

L'objet du rapport consiste à exposer en détail la manière dont plusieurs vulnérabilités peuvent être utilisées pour contrôler et affaiblir la sécurité d'applications web. Nous avons choisi d'en traiter 3 : les Uploads non sécurisés, la gestion des contrôles d'accès et le cryptage des mots de passe.

Uploads non sécurisé

Dans le contexte d'applications Web, la fonctionnalité de téléversement de fichiers représente à la fois une commodité essentielle pour l'utilisateur final et une porte d'entrée privilégiée pour des agents malveillants. Lorsqu'un site s'appuie sur une pile Node.js, le danger n'est pas tant l'exécution directe du code chargé (Node n'interprétera pas un fichier *.php* nativement) mais la multiplicité des chemins détournés susceptibles d'affecter gravement l'infrastructure.

Un simple fichier type « shell déguisé » (*shell.php*) ou tout autre artefact apparemment anodin peut être exploité pour franchir les barrières de sécurité d'une application Node.js.

Partie 1 : mécanismes d'exploitation d'un fichier malveillant dans un environnement Node.js

Lorsqu'un agent malveillant réussit à envoyer un fichier comme *cv.pdf.php* dans un dossier accessible depuis Internet, plusieurs types d'attaques sont possibles. Le scénario le plus simple se produit si le serveur Node.js partage son espace de stockage avec un serveur PHP, comme Apache. Dans ce cas, une simple visite du fichier via un lien peut lancer le script PHP et donner à l'agent un accès direct au système : il peut alors naviguer dans les dossiers, lire des fichiers sensibles comme les clés *JWT_SECRET*, ou encore accéder aux identifiants de services cloud (si existant – ici non). Mais d'autres attaques, plus discrètes, existent aussi. Un fichier SVG, par exemple, peut contenir du code JavaScript caché. Si ce fichier est affiché tel quel dans une image sur le site, le script s'exécute dans le navigateur d'un autre utilisateur et peut voler sa session (attaque XSS). De même, un document Word ou Excel peut contenir des macros ou des liens vers des fichiers malveillants, qui se déclenchent lors de l'ouverture ou lors d'un traitement automatique. D'autres attaques encore utilisent des outils installés sur le serveur comme ImageMagick ou LibreOffice. Si un pirate téléverse un fichier piégé, ces outils peuvent être détournés pour exécuter du code à distance, même si l'application Node.js ne fait que lancer une commande. Enfin, il existe des attaques qui ne nécessitent même pas d'exécution de code. Par exemple, un fichier très compressé peut devenir énorme une fois décompressé, ce qui remplit la mémoire ou le disque et fait planter le serveur.

D'un autre côté, ce rapport détaille aussi la stratégie défensive que nous avons mise en place pour faire face à ces risques. Elle repose sur un ensemble cohérent de protections techniques, visant à bloquer ou à limiter chacune de ces tentatives, tout en assurant la sécurité globale du système.

Partie 2 : stratégie de mitigation intégrée dans l'application

Pour réduire de manière significative la surface d'attaque, nous avons d'abord déplacé le dossier *uploads* hors de toute racine statique, supprimant ainsi la possibilité d'atteindre les fichiers par URL directe ; seule certaines routes POST du fichier *users.jsy* possède un accès en écriture, tandis que les permissions POSIX restreignent toute lecture à un groupe système dédié. En amont, Multer a été configuré en *diskStorage* afin que chaque fichier soit renommé selon un identifiant UUID v4, éliminant non seulement le risque de collision mais aussi l'induction de logique métier à partir du nom original. Ce middleware refuse désormais toute charge dont l'extension ou le *mimetype* n'appartient pas au périmètre *.pdf*, *.jpeg*, *.png*, *.xlsx* ou *.docx*. En outre, la limite de taille fixée à un mégaoctet coupe court aux flux volumineux avant leur écriture sur disque, freinant les tentatives de saturation de ressources. Une fois la copie terminée, un module interne, *security.js*, lit le contenu : l'extraction textuelle des DOCX s'effectue au moyen de Mammoth, celle des feuilles Excel via la bibliothèque *xlsx*, tandis que le reste est traité en lecture binaire. Sur ce texte ou ce flux, une expression régulière détecte la présence de marqueurs suspects tels que *<?php*, *shell_exec*, *eval*, *base64_decode* ou *system*, et supprime immédiatement le fichier tout en renvoyant un code 422 que la couche View affiche sous forme d'alerte explicite. À cette défense primaire s'ajoutent plusieurs garde-fous : un démon ClamAV analyse de façon asynchrone chaque nouvelle entrée afin d'identifier des signatures connues, un *cron* de maintenance expurge les fichiers temporaires vieux de plus de quarante-huit heures, et les contenus finalement acceptés sont mis à disposition des utilisateurs par des URL pré-signées S3, empêchant toute exécution locale ultérieure. Couplées à une supervision Prometheus qui déclenche des alertes sur toute croissance anormale de l'espace occupé, ces mesures construisent une défense en profondeur où, à chaque étape, l'adversaire doit franchir une barrière supplémentaire, rendant la compromission beaucoup moins probable et maintenant l'application conforme aux recommandations OWASP pour la gestion des fichiers téléversés.

Mots de passe non haché

Dans une application web moderne, la gestion des identifiants utilisateur constitue un point de vigilance critique. Le mot de passe représente une cible privilégiée pour tout attaquant. Lorsqu'un site est développé avec Node.js, le danger n'est pas seulement celui d'un vol de mot de passe stocké en clair, mais celui, plus vaste, d'une compromission globale de comptes, de sessions ou de privilèges administrateur en cas d'attaque réussie. Un simple accès en lecture à la base de données suffit à révéler la faiblesse d'un système si les mots de passe y sont conservés sans protection ou avec un chiffrement réversible.

Partie 1 : Risques liés à un mauvais chiffrement des mots de passe dans une application Node.js

Si un mot de passe est enregistré tel quel dans une base de données, ou même s'il est chiffré par un algorithme réversible (comme AES ou base64), un simple vol de la base, que ce soit via une faille d'injection SQL, une élévation de privilège ou une fuite de sauvegarde, suffit à exposer l'ensemble des comptes utilisateurs. Dans le cas d'un stockage en clair, aucune étape supplémentaire n'est nécessaire pour l'attaquant : il accède instantanément à tous les identifiants. Même dans le cas d'un chiffrement réversible, le vol de la clé de déchiffrement permet de retrouver les mots de passe d'origine, en quelques secondes. Ce type de vulnérabilité est d'autant plus grave que de nombreux utilisateurs réutilisent leurs mots de passe sur plusieurs sites. Une base compromise sur un site peu protégé devient alors un tremplin vers des services plus sensibles, tels que des boîtes mails, des réseaux sociaux ou des plateformes de paiement.

Au-delà du vol direct, une autre attaque courante consiste à tester automatiquement des millions de mots de passe possibles (attaque par force brute ou dictionnaire). Si les mots de passe sont simplement hachés avec des algorithmes rapides comme SHA-1 ou MD5, l'attaquant peut les casser à l'aide de bases pré-construites (rainbow tables) ou de GPU optimisés.

Enfin, si le hachage est mal implémenté, par exemple si le même mot de passe donne toujours le même résultat, alors deux utilisateurs différents ayant le même mot de passe auront un hachage identique, ce qui permet de repérer immédiatement les doublons et de mener des attaques ciblées. La sécurité des mots de passe ne repose donc pas seulement sur l'interdiction du stockage en clair, mais aussi sur le choix des bons algorithmes de hachage, la gestion des paramètres de dérivation, et une stratégie de contrôle d'accès rigoureuse.

Partie 2 : Stratégie de protection des mots de passe mise en place dans l'application

Afin d'éliminer toute possibilité de fuite exploitable, nous avons mis en place une stratégie de sécurisation des mots de passe fondée sur un hachage robuste et irréversible. Concrètement, chaque mot de passe est traité via la bibliothèque *bcrypt*, largement reconnue pour sa résistance aux attaques par force brute. Lors de l'enregistrement d'un compte, l'application génère un 'sel' unique pour chaque utilisateur, puis calcule le hachage du mot de passe avec un facteur de coût de 10, ce qui ralentit volontairement le processus de calcul afin de rendre impossible un traitement massif par un attaquant. Cette opération est réalisée de manière asynchrone, afin de ne pas bloquer le thread principal de l'application Node.js. Aucune information n'est stockée en clair, et même les administrateurs de la base de données ne peuvent retrouver un mot de passe d'origine. Lors de la connexion, le mot de passe saisi par l'utilisateur est comparé à l'empreinte en base à l'aide de la méthode *bcrypt.compare()*, qui garantit l'usage du même 'sel' et du même algorithme que lors de l'enregistrement.

Enfin, les contrôles d'accès sont mis en place pour que seules certaines routes puissent interagir avec les identifiants utilisateurs, et les erreurs en cas de

tentative de connexion échouée sont volontairement vagues, afin de ne pas révéler si un identifiant ou un mot de passe est incorrect.

L'ensemble de ces mesures permet de garantir qu'un mot de passe volé ou intercepté dans la base reste inutilisable, qu'un haché identique ne puisse être répliqué, et que la sécurité de l'ensemble du système repose sur une architecture défensive conforme aux recommandations OWASP.

Contrôle d'accès

Finalement, dans le cadre d'une application de recrutement, les utilisateurs interagissent avec le système selon des rôles distincts, candidat, recruteur ou administrateur, et chacun ayant accès à des fonctionnalités bien délimitées. Pourtant, sans un contrôle rigoureux des autorisations associées à chaque rôle, des utilisateurs malveillants ou simplement curieux peuvent facilement accéder à des sections qui ne leur sont pas destinées. Sur une application Node.js, le risque n'est pas théorique : si un recruteur accède par exemple à une route réservée à un administrateur ou si un candidat parvient à consulter les profils d'autres utilisateurs, l'ensemble du système perd en intégrité, en confidentialité et en confiance.

Partie 1 : Risques liés à l'absence de filtrage des routes par rôle dans une application Node.js

Dans de nombreuses architectures web, les rôles sont définis côté base de données ou lors de la création de session, mais cela ne garantit pas qu'ils soient effectivement vérifiés lors de chaque requête. En l'absence de vérification systématique, un utilisateur peut simplement forger une URL vers une section du site à laquelle il n'a pas été invité. Un candidat connecté peut ainsi essayer d'accéder manuellement à `/admin` ou `/recruteur/123456789`, espérant que l'interface ou les contrôleurs backend n'aient pas mis en place de protection efficace. Dans certains cas, des routeurs mal conçus peuvent même renvoyer des données sensibles ou des formulaires d'édition à n'importe quel utilisateur connecté, sans s'assurer que celui-ci détient les autorisations requises.

Le risque est aggravé lorsque les identifiants d'accès aux données sont intégrés dans l'URL (par exemple un numéro SIREN dans `/recruteur/123456789`), car l'utilisateur peut alors modifier manuellement ces paramètres pour consulter ou modifier les données d'un autre compte.

Si un tel comportement est possible, il suffit que l'utilisateur intercepte une requête ou tente différentes adresses pour accéder à des informations personnelles, modifier des offres d'emploi, ou consulter des candidatures privées. Ce type de faille remet directement en cause la confidentialité des données, la séparation des rôles, et la responsabilité du service vis-à-vis de ses utilisateurs. Il en résulte un risque juridique et éthique majeur, notamment dans le cadre du RGPD. Un système qui ne contrôle pas l'accès aux routes sensibles est vulnérable à des attaques internes (utilisateur légitime accédant à des données non autorisées), bien plus difficiles à détecter qu'une attaque extérieure.

Partie 2 : Stratégie de contrôle d'accès par rôle mise en place dans l'application

Pour garantir une séparation stricte des permissions entre les différents types d'utilisateurs, nous avons mis en œuvre une fonction centrale `hasAccess`, appelée systématiquement avant toute action sensible. Cette fonction s'appuie sur la session utilisateur pour vérifier trois critères essentiels : la connexion effective, le rôle déclaré, et la cohérence entre le chemin demandé et les autorisations associées à ce rôle.

Dans un premier temps, la fonction vérifie que l'utilisateur dispose bien d'un identifiant de session valide, condition préalable à toute action. Ensuite, elle compare le rôle stocké dans la session à une liste de chemins autorisés. Chaque rôle dispose de ses propres préfixes de routes : l'administrateur ne peut accéder qu'à `/admin`, le recruteur aux routes de type `/recruteur`, et le candidat aux zones `/users` et `/candidat`.

Le cas du recruteur fait l'objet d'une vérification supplémentaire : pour accéder à une route personnalisée de type `/recruteur/{siren}`, la fonction extrait le numéro SIREN depuis l'URL et vérifie qu'il correspond bien à celui stocké dans la session. Cela empêche un recruteur d'accéder aux données d'une autre entreprise simplement en changeant l'URL.

Cette logique de filtrage est indépendante de l'interface utilisateur, ce qui empêche un accès non autorisé même en cas de manipulation directe des URL. En cas d'échec, l'accès est immédiatement refusé, et aucune donnée n'est retournée. La sécurité est donc assurée côté serveur, là où elle ne peut être contournée. Cette approche garantit que la structure même du site repose sur une logique d'autorisation solide et facilement maintenable. L'application respecte ainsi les grands principes de contrôle d'accès recommandés par l'OWASP et garantit l'étanchéité entre les différentes sphères d'utilisateur.